

Volet C : Data Scientist

Le besoin métier de Néovolt est :

prédire la consommation future afin d'anticiper les achats d'énergie.

Le modèle répond donc à la question :

"Combien d'énergie allons-nous consommer demain ?"

1. Import des librairies

J'importe des bibliothèques Python qui me permettent de manipuler les données, faire des calculs et afficher des graphiques.

- **pandas et numpy** : pour la manipulation et le traitement des données.
- **matplotlib** : pour la visualisation des résultats.
- **scikit-learn** : pour la construction et l'évaluation des modèles de machine learning.
- **joblib** : pour la sauvegarde du modèle entraîné afin de pouvoir le réutiliser dans une API.

Ensuite, plusieurs modèles de régression sont initialisés :

- **LinearRegression** : modèle simple qui suppose une relation linéaire entre les variables explicatives et la consommation.
- **RandomForestRegressor** : modèle basé sur des arbres de décision, capable de capturer des relations complexes et non linéaires.
- **GradientBoostingRegressor** : modèle plus avancé qui améliore progressivement ses performances en corrigeant les erreurs des modèles précédents.

2. Chargement des données

On lit les fichiers CSV, je charge les données de consommation, météo et compteurs dans des DataFrames pour pouvoir les analyser.

```
df_conso = pd.read_csv("donnees/relevés_consommation.csv")
```

```
df_meteo = pd.read_csv("donnees/meteo.csv")
```

```
df_compteurs = pd.read_csv("donnees/compteurs.csv")
```

1. df_conso (relevés_consommation.csv)

Cette table contient les **relevés de consommation électrique des compteurs**.

Colonnes utilisées :

- date → permet l'analyse temporelle
- consommation_kwh → variable cible à prédire (la consommation d'énergie)

2. df_meteo (meteo.csv)

Cette table contient les **données météorologiques** associées aux mêmes dates. La consommation d'énergie dépend fortement de la météo (chauffage en hiver, climatisation en été).

Colonnes utilisées :

- date → pour la fusion avec les autres tables
- temp_moyenne_c → température moyenne (feature principale)
- temp_min_c → température minimale
- temp_max_c → température maximale

3. df_compteurs (compteurs.csv)

Colonnes (selon projet) :

- id_pdl identifiant)
- zone
- type de compteur

Elle permettrait de segmenter la consommation par zone ou par type de compteur.

conclusion : les données utilisées proviennent de trois sources complémentaires : les relevés de consommation électrique, les données météorologiques et les informations sur les compteurs. Les données de consommation représentent la

variable cible du modèle, tandis que les données météo sont utilisées comme variables explicatives afin d'améliorer la précision des prédictions. Les données des compteurs permettent une ouverture vers des analyses plus avancées par zone ou type de compteur.

3. Nettoyage des données

- Conversion des dates : je convertis les dates pour pouvoir faire des analyses temporelles. transforme texte → vraie date

```
df_conso["date"] = pd.to_datetime(df_conso["date"])
```

```
df_meteo["date"] = pd.to_datetime(df_meteo["date"])
```

- Suppression des doublons : Je supprime les doublons pour éviter de compter deux fois la même consommation.

```
df_conso = df_conso.drop_duplicates()
```

- Supprimer les valeurs négatives : Je supprime les valeurs négatives car elles ne sont pas réalistes.

```
df_conso = df_conso[df_conso["consommation_kwh"] >= 0]
```

- Gérer les valeurs manquantes : Je remplis les valeurs manquantes pour éviter d'avoir des trous dans les données.

```
df_conso["consommation_kwh"] =  
df_conso["consommation_kwh"].interpolate()
```

```
df_meteo = df_meteo.ffill()
```

- ❖ interpolation → remplit avec logique
- ❖ ffill → reprend la valeur précédente

Vérification : vérifier qu'il n'y a plus de données manquantes

```
print(df_conso.isnull().sum())
```

4. Préparation des données

- Je regroupe les données par jour afin d'obtenir la consommation totale quotidienne. En effet, les relevés de consommation peuvent être enregistrés plusieurs fois par jour et pour plusieurs compteurs. L'agrégation permet donc d'obtenir une valeur unique par jour, plus facile à analyser et à utiliser pour la modélisation.

```
df_daily =  
df_conso.groupby("date")["consommation_kwh"].sum().reset_index()
```

- Moyenne des températures par jour. On regroupe les données par jour pour avoir une seule valeur de consommation et une seule valeur météo par jour. Ça permet de construire une série temporelle propre pour entraîner le modèle.

```
df = pd.merge(df_daily, df_meteo_daily, on="date", how="left")
```

- Je fusionne les données pour enrichir la consommation avec des informations météo.

On combine consommation + météo, **relier la consommation d'énergie avec les conditions météo**, parce que la météo influence directement la consommation

```
df = pd.merge(df_daily, df_meteo_daily, on="date", how="left")
```

Tableau de consommation on a : date, consommation_kwh

Tableau météo on a : date, température moyenne, température min, température max

Ces deux tableaux parlent du même jour, mais ils ne sont pas ensemble.

Pour que ton modèle puisse apprendre :

“Quand il fait froid, la consommation augmente” ou “Quand il fait chaud, la consommation baisse” il faut mettre les deux informations sur la même ligne.

5. Création des variables : afin d'améliorer la capacité du modèle à prédire la consommation énergétique, plusieurs variables explicatives ont été créées. Ces variables permettent d'intégrer des informations temporelles, comportementales et

météorologiques qui influencent directement la consommation. L'objectif est d'enrichir la série temporelle avec des signaux pertinents pour aider le modèle à mieux comprendre les variations naturelles de la demande

- Variables temporelles :

```
df["jour_semaine"] = df["date"].dt.dayofweek
```

```
df["mois"] = df["date"].dt.month
```

Explication : La consommation n'est pas la même selon le jour ou la période de l'année.

- Les jours ouvrés ont une activité plus élevée.
- Les week-ends et jours fériés ont une consommation plus faible.
- Les mois reflètent la saisonnalité (chauffage en hiver, climatisation en été).

- Variable "weekend"

```
df["weekend"] = (df["jour_semaine"].isin([5,6])).astype(int)
```

Explication :

Le comportement énergétique est différent le week-end : moins d'activité, moins de machines, moins de présence → consommation plus faible.

Cette variable aide le modèle à distinguer jours ouvrés / non ouvrés.

- Variables de décalage (lags)

```
df["lag_1"] = df["consommation_kwh"].shift(1) df["lag_7"] =  
df["consommation_kwh"].shift(7)
```

Explication : La consommation d'un jour dépend souvent de celle des jours précédents.

- *lag_1* = consommation de la veille
- *lag_7* = consommation du même jour la semaine précédente

J'utilise la consommation des jours précédents pour prédire la consommation future.

- Moyenne mobile

```
df["rolling_7"] = df["consommation_kwh"].rolling(7).mean().shift(1)
```

Explication : La moyenne mobile sur 7 jours permet de lisser les variations quotidiennes et de capturer la tendance générale de la semaine.

Cela stabilise la série et aide le modèle à mieux comprendre les cycles.

- Température

```
df["temp"] = df["temp_moyenne_c"]
```

Explication : La météo influence directement la consommation :

- froid → chauffage → consommation augmente
- chaleur → climatisation → consommation augmente
- températures douces → consommation plus faible

C'est l'une des variables les plus importantes dans un modèle énergétique.

Plusieurs variables explicatives ont été créées afin d'enrichir la série temporelle et d'améliorer la qualité des prévisions. Des variables temporelles (jour de la semaine, mois, week-end) ont été ajoutées pour capturer les effets calendaires et la saisonnalité. Des variables de décalage (lag_1, lag_7) permettent d'intégrer l'influence de la consommation passée, tandis qu'une moyenne mobile sur 7 jours (rolling_7) aide à lisser les fluctuations quotidiennes. Enfin, la température moyenne a été intégrée, car les conditions météorologiques ont un impact direct sur la consommation énergétique. L'ensemble de ces variables fournit au modèle des informations essentielles pour mieux comprendre et anticiper les variations de consommation.

6. Suppression des lignes incomplètes : enlever lignes avec données manquantes

```
df = df.dropna()
```

7. Visualisation : Voir l'évolution de la consommation dans le temps. Une première visualisation de la série temporelle permet d'observer l'évolution de la consommation

au fil des jours

```
plt.plot(df["date"], df["consommation_kwh"])
```

8. Modèle

- Features : Variables d'entrée.

```
features = [...]
```

Les features correspondent aux variables explicatives utilisées par le modèle pour prédire la consommation. Elles regroupent les informations temporelles, les données météo et les indicateurs dérivés (lags, moyennes mobiles).

- Target : Ce qu'on veut prédire.

```
target = "consommation_kwh"
```

La variable cible (target) est la consommation énergétique quotidienne, que le modèle doit apprendre à estimer.

- Split

```
train = apprendre
```

```
test = vérifier
```

Les données sont séparées en deux ensembles :

- **train** : utilisé pour entraîner le modèle
- **test** : utilisé pour évaluer sa capacité à généraliser

Le découpage est temporel afin d'éviter toute fuite d'information entre passé et futur.

9. Entraînement

```
model = RandomForestRegressor(...) model.fit(X_train, y_train)
```

Le modèle Random Forest est entraîné sur l'ensemble d'apprentissage. Il apprend les relations entre les variables explicatives (features) et la consommation réelle afin de pouvoir effectuer des prédictions fiables.

10. Prédictions

```
predictions = model.predict(X_test)
```

Une fois entraîné, le modèle génère des prédictions sur l'ensemble de test, ce qui permet de comparer les valeurs estimées avec les valeurs réelles.

11. Évaluation

- MAE → erreur moyenne
- RMSE → erreur plus sévère
- R^2 → qualité globale

Plusieurs métriques sont utilisées pour évaluer les performances du modèle :

- **MAE** (Mean Absolute Error) : mesure l'erreur moyenne en kWh.
- **RMSE** (Root Mean Squared Error) : pénalise davantage les grosses erreurs.
- **R^2** : indique la proportion de la variabilité expliquée par le modèle.

Ces indicateurs permettent de juger la précision globale du modèle et sa capacité à reproduire les variations de consommation.

Le modèle atteint un R^2 de 0.8894, ce qui signifie qu'il explique près de 89 % de la variabilité de la consommation. Ce niveau de performance traduit une très bonne capacité de prédiction pour un modèle basé sur des données réelles, avec une erreur moyenne (MAE) de 2638 kWh et un RMSE de 4261 kWh. Les résultats montrent que le modèle capture efficacement la tendance générale et les variations quotidiennes de la consommation.

Comparaison de modèles

Donc on va comparer 3 modèles :

- Régression Linéaire
- Random Forest
- Gradient Boosting

Trois modèles ont été testés afin d'évaluer leur capacité à prédire la consommation énergétique quotidienne : une régression linéaire, un Random Forest et un Gradient Boosting.

La régression linéaire sert de modèle de référence (baseline). Elle obtient un R^2 de 0.883, ce qui montre qu'elle capture une partie importante de la tendance générale, mais ses performances restent limitées. Cela indique que la relation entre les variables explicatives et la consommation n'est pas strictement linéaire.

Le Random Forest améliore légèrement les résultats ($R^2 = 0.889$). Grâce à sa capacité à modéliser des relations non linéaires et des interactions entre variables, il réduit le RMSE et offre une meilleure stabilité.

Le Gradient Boosting est le modèle le plus performant, avec un R^2 de 0.895 et les plus faibles MAE et RMSE. Ce résultat est cohérent avec son fonctionnement basé sur l'optimisation séquentielle des erreurs, qui lui permet de mieux capturer les variations fines de la consommation.

Les trois modèles présentent de bonnes performances avec un coefficient de détermination supérieur à 0,88. Le modèle Gradient Boosting obtient les meilleurs résultats avec un MAE de 2515 kWh, un RMSE de 4151 kWh et un R^2 de 0,895. Il a donc été retenu comme modèle principal pour la prévision de consommation.

Analyse critique

Bien que le modèle Random Forest atteigne un R^2 de 0.889, ce qui traduit une très bonne capacité de prédiction, plusieurs limites doivent être soulignées :

- Le modèle a tendance à lisser les pics de consommation, ce qui peut être problématique pour des usages critiques (achat d'énergie, dimensionnement).
- Il ne prend pas en compte les jours fériés, vacances scolaires ou événements exceptionnels, qui peuvent fortement influencer la consommation.
- Les performances dépendent de la qualité des données météo, qui peuvent contenir du bruit ou des imprécisions.
- Le Random Forest ne capture pas les dépendances temporelles longues, contrairement à des modèles séquentiels (LSTM, GRU).

Une sous-estimation importante pourrait entraîner un sous-achat d'énergie, tandis qu'une surestimation pourrait générer un surcoût. Ces risques doivent être pris en compte dans une mise en production réelle.

MLOps

Le MLOps consiste à organiser et automatiser le cycle de vie d'un modèle de Machine Learning : de son développement jusqu'à son déploiement et son suivi.

Le projet est versionné sur GitHub, permettant de suivre les modifications et d'assurer la traçabilité du développement.

Le code source est versionné dans GitHub afin d'assurer la traçabilité des évolutions. Chaque membre de l'équipe travaille sur sa propre branche avant fusion dans la branche principale.

Le modèle est sauvegardé dans un fichier afin de pouvoir être réutilisé sans nécessiter un nouvel entraînement. Cette étape est essentielle pour le déploiement du modèle en production.

```
joblib.dump(best_model, "models/gradient_boosting_v1.pkl")
```

Conclusion :

Le MLOps regroupe l'ensemble des pratiques permettant de gérer le cycle de vie du modèle, de son développement à son utilisation en production.

Dans ce projet, cela inclut :

- le versioning du code avec GitHub
- la reproductibilité du modèle
- la sauvegarde du modèle
- le suivi des performances
- la possibilité de réentraînement

Ces éléments permettent d'assurer la fiabilité et la durabilité du modèle dans le temps.

Déploiement

Sources de données

(Compteurs + Météo)



Collecte



Pipeline de préparation

- nettoyage

- transformation

- feature engineering



Modélisation

(Gradient Boosting)



Prédiction

(Consommation énergétique)



Exploitation métier

(Dashboard, aide à la décision)